The RustRed reduction path

Where equal integrals meet, what “fully reduced” means, and what a cache remembers.

Source snapshot: Native `4190f2c6` and Vakint `d29eca063` · 26 October 2026. This describes the existing default path. The examples are schematic exact identities, not measured reductions.

There are intermediate merges

RustRed does not postpone every addition until the end. It merges equal keys at several boundaries, but each boundary sees a different object. The distinction is between **equal immediate children** and **equal descendants hidden inside separate branches or roots**.

1 **Admit and collect the input roots**
Validate every incoming key and coefficient, retain its conditions, then add coefficients of identical roots. Exact zero entries disappear. Thus $aI + bI \to (a+b)I$ before recursion, while admission and denominator checks still apply even if $a + b = 0$.

↓

2 **Evaluate one rule; collect its immediate children**
Specialize the selected formula, check its original denominators and descent, then combine equal RHS child keys. A local RHS $U - U + V$ becomes V; that occurrence of U needs no recursive expansion.

↓

3 **Finish children by depth-first traversal; combine their results**
A child is solved or supplied by a completed cache entry. Once all surviving children are ready, back-substitute their terminal maps, multiply by the edge coefficients, and add equal terminal keys progressively. Exact zeros disappear at this level too. Store the completed parent result.

↓

4 **Collect one completed root into the request output**
Multiply that root’s terminal map by its external input coefficient, then add it to the running output. Continue with the next root. Cross-root cancellation happens here, after each contributing root has been terminal-expanded.

What “fully reduces a root” means

For a successful strict reduction, all required descendants have been replaced by declared Native stopping terminals or sealed zeros, with exact parameter-dependent coefficients and the required nonzero conditions. A completed cached child can replace its entire recursive computation. “Fully” describes the **result**, not an instruction to recompute every descendant, and not evaluation of a Laurent series or a numerical master integral. Coefficients may retain a factorized internal representation: terminal completion does not require expanding every rational-polynomial product.

The parent combination waits for its children to finish. It does not continuously pool the not-yet-reduced descendants of every branch. Optional preserve mode can leave undeclared integrals explicit; a ready partial result is not strict terminal completion and is not eligible as a distributed completed identity.

Source anchors. Native `core/solver/candidate_reduction/evaluator.rs:250–302`; `core/solver/candidate_reduction/routed/application/engine.rs:877–1000,1070–1124,1387–1417`. Here and below `core/` means `crates/rustred-core/src/`; `app/` means `crates/rustred-app/src/`.

A cancellation can be real and still arrive late

Let $M_1, \ldots, M_4$ denote four declared stopping objects in this example. Assume the selected exact rules are

$X \to 2U + V$
$U \to M_1 + M_2$
$W \to -M_1 + M_4$
$Y \to -2U + W$
$V \to M_1 - M_3$
input: $X + Y$

Current traversal: completed child results first

Step	Work / available value	Collected result
Solve X	Finish U and V; combine $2U + V$.	$3M_1 + 2M_2 - M_3$
Solve Y	Reuse completed U if still resident; finish W; combine $-2U + W$.	$-3M_1 - 2M_2 + M_4$
Collect	Add the two fully expanded root contributions.	$-M_3 + M_4$

The coefficients of M_1 and M_2 cancel exactly. Nevertheless U was needed while computing X. Under the stated residency assumption, it is computed **once** and later reused: cancellation of its final contribution does not mean its expansion was avoided. With finite eviction, a later miss may recompute it; an admitted persistent hit can also supply the completed value.

Local cancellation — implemented
$R \to U - U + V \to V$
The identical child keys meet in one rule's RHS collection. No recursive work for these cancelled occurrences of U is scheduled. Checks attached to the selected formula are still retained.

A global weighted frontier — hypothetical
$X + Y \to 2U + V - 2U + W$
Pool these descendants first: $2U - 2U = 0$, then solve only $V + W$. The current default traversal does not perform this pooling.

The hypothetical path would avoid U's work, but would then have no completed unit mapping for U, X or Y to cache without additional work. Actually completed V and W mappings could be cached. The current path instead produces reusable mappings for U, X and Y; a weighted partial/global sum must never be stored as one of those individual identities.

A diamond within one root has the same distinction

If $R \to A + B$, $A \to U + V$ and $B \to -U + W$, A and B are different immediate children. RustRed finishes their results before combining them at R. Shared U can be reused from cache, but the $+U$ and $-U$ hidden inside different branches are not first merged into a common unreduced frontier.

The notation hides the operational keys

Our I, U and M_i suppress family and routing detail. Native actually memoizes **Route(key)** and **Apply(owner, key)** separately. A same-owner child remains Apply; a proper pinched support can return to Route for transport to its owner. An equal vector of integral powers alone does not make those two states interchangeable. Every stored expansion is for the **unit node**: the external coefficient a of aI belongs to the calling edge/root, not to the cached identity for I.

Source anchors. Native `core/solver/candidate_reduction/cache.rs:149–179`; `core/solver/candidate_reduction/routed/trace.rs:12–16,89–108`; `core/solver/candidate_reduction/routed/application/engine.rs:983–1000,1085–1124`. The example assumes no eviction of the completed shared node between its uses.

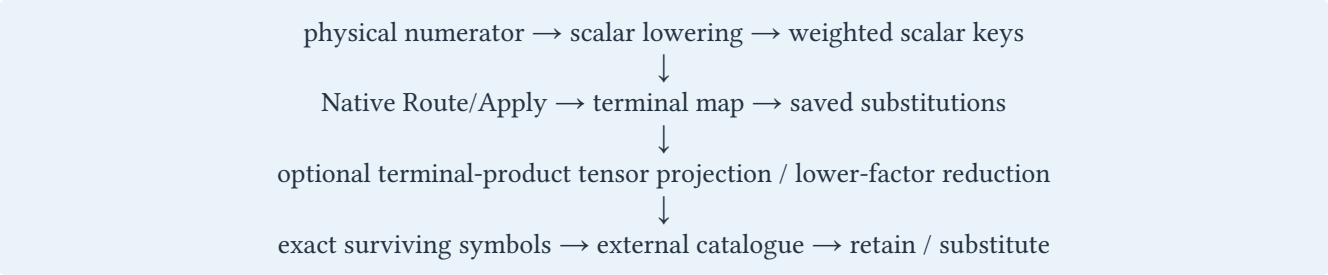
What is remembered, and when it becomes a file

Layer	How it is generated	What it means
In-memory native memoization	DFS creates completed terminal maps for leaves and combined parents; later roots can reuse completed descendants.	Exact unit-node expansion and its guards. Bounded policies may evict completed entries; live parent dependencies are pinned.
Persistent backing / capture	Preparation/export or an attached capture sink validates eligible completed records. A backing supplies previously generated records on demand.	Exact unit-node coefficients and guards, bound to operational keys and the authenticated package. No pending stack or weighted partial request sum.
Running physical-example cache	When attached, normal reductions forward completed descendants to a bounded writer while the larger request runs. Backing reads and captures are separate.	A durable prefix can survive an unfinished root/workload. Flush/snapshot/close establish I/O barriers; a closed writer does not certify workload completion.
Publication compressed cache	A separate export/bundle/archive step packages selected or cumulative records for transport.	Compression and selection create no new identities. Hosting the artifact is separate from producing it and validating the workload.

An entry means $I = \sum_i c_i M_i$ under retained conditions $g_j \neq 0$. The persistent binding covers the source package, routes, entry scope, zero authority and terminal substitutions, plus family, arity and solver layer. Loads check the binding and current policy; coefficients and conditions are kept together. A coefficient cancelling to zero does not erase the original guards of an evaluated formula or imported subtree.

Unresolved leaves are excluded from distributed completed records. Preserve mode can memoize a partial result locally. Capture/export checks its remaining keys and skips it if any is undeclared to the bound Native application. An exactly cancelled parent can itself be complete and exportable even when a partial child's identity is excluded.

From Vakint's numerator to Native terminals and the master catalogue



Scalar lowering creates weighted scalar keys before Native reduction. The optional `factor_projection` callback is later: it projects tensor numerators in terminal product charts, retaining projector denominator conditions **before** materializing and simplifying their sum. Native stopping terminals and saved substitutions are internal conventions; their final family-qualified output map is not the external catalogue itself.

After exact combination of the resulting master symbols, Vakint distinguishes known coefficients, declared-but-uncomputed masters, bare Native announcements, and undeclared residuals. The 640-entry five-loop campaign declares external masters with uncomputed coefficients; it is not a claim that Native has 640 leaves or that those masters have been evaluated. This classification precedes optional master-coefficient substitution. Vakint's topology matching at ingress is a different use of a catalogue.

Source anchors. Native `core/solver/candidate_reduction/routed/application/persistent.rs:71–101,291–344,401–460,504–546`; `app/application/saved_application/mod.rs:433–476,532–607`; `app/application/saved_application/running_cache.rs:435–456,689–718`. Vakint `src/rustred_evaluation/factor_projection.rs:90–113`; `cache_five_loop.rs:299–419`; `cache_running.rs:20–33,234–256`; `five_loop.rs:488–527,889–967,1006–1021`; `masters.rs:513–563,644–655`; `data.rs:1149–1178`. Revisions: Native `4190f2c6f3158e3fc3cbcb1937e12e5971da9175`; Vakint `d29eca063a33690c8395a8e0c34dca869847f987`.